

Հայաստանի Ազգային Պոլիտեխնիկական
Համալսարան



Առարկա: Ծրագրային անվտանգություն
Խումբ: SS-355 (Լաբ.-1)

Ուսանող: Մելիքյան Վերոնիկա

Outlook: veronikamelikyan.tt355@polytechnic.am

ԹԵՄԱ 1 — Ֆունկցիայի Արգումենտների Թվի Խեղաթյուրում

Ի՞նչ է դա

Սա կոդի OBFUSCATION (թաքցնում) մեթոդ է, որտեղ ֆունկցիան ունի ավելի շատ կամ ավելի քիչ արգումենտ, քան իրականում օգտագործում է: Նպատակն է շփոթեցնել ծրագրի վերլուծողին (reverse engineer-ին):

Ե՞րբ է կիրառվում

- Malware-ներում — վնասակար ծրագրերում
- License check-երում — լիցենզիայի ստուգման կոդում
- Crackme-ներում — հատուկ «կոտրելու» ծրագրերում
- Obfuscated software-ներում

Մեթոդ 1 — Dummy Parameters (Կեղծ Արգումենտներ)

Ֆունկցիան ունի «կեղծ» (dummy) արգումենտներ, որոնք ոչ մի կերպ չեն օգտագործվում: Reverse engineer-ը տեսնում է ֆունկցիայի ստորագրությունը և ժամանակ է կորցնում վերլուծելու, թե ինչ են անում կեղծ արգումենտները:

C++ Օրինակ

```
int obf_add(int a, int b, int fake1, int fake2) {  
    // fake1 and fake2 are NEVER used  
    return a + b;  
}  
  
int main() {  
    // Calls with garbage values - confuses analyst  
    cout << obf_add(5, 3, 999, 12345) << endl; // prints 8  
}
```

Ի՞նչ է տեղի ունենում: obf_add(5, 3, 999, 12345) կանչի ժամանակ

a = 5, b = 3 -> օգտագործվում են

fake1 = 999, fake2 = 12345 -> ԱՆՏԵՍՎՈՒՄ են

Ֆունկցիան վերադարձնում է $5 + 3 = 8$

Reverse engineer-ը տեղ է ծախսում fake1/fake2 վերլուծելու վրա

Python Օրինակ

```
def obf_add(a, b, *args): # *args – ցանկացած թիվ լրացուցիչ արգ.  
    # args is a tuple but completely ignored  
    return a + b  
  
print(obf_add(5, 3, 999, 888, 777)) # prints 8
```

Python-ում *args-ն «կուլ» է տալիս բոլոր ավելցուկ արգումենտները՝ ոչ մի error չառաջացնելով: Ֆունկցիան աշխատում է կարծես ոչինչ չի եղել:

Մեթոդ 2 — Packed Parameters (Փաթեթավորված Արգումենտներ)

Երկու կամ ավելի արժեք «փաթեթավորվում» են մեկ ամբողջ թվի մեջ բիտային գործողություններով (bitwise operations): Ֆունկցիան ստանում է 1 արգումենտ, բայց ներսում «ապաքողարկում» է 2+ արժեք:

Բիտային Փաթեթավորում — Բացատրություն

`a = 5 = 0000 0000 0000 0101` (16 բիտ)

`b = 3 = 0000 0000 0000 0011` (16 բիտ)

`packed = (a << 16) | b`

`= 0000 0000 0000 0101 0000 0000 0000 0011` (32 բիտ)

`= 327683` (decimal)

Ապաքողարկում:

`a = (packed >> 16) & 0xFFFF -> 5`

`b = packed & 0xFFFF -> 3`

C++ Օրինակ

```
int obf_add(int packed) {  
    int a = (packed >> 16) & 0xFFFF; // Upper 16 bits -> a  
    int b = packed & 0xFFFF; // Lower 16 bits -> b  
    return a + b;  
}  
  
int main() {  
    int a = 5, b = 3;  
    int packed = (a << 16) | b; // 327683  
    cout << obf_add(packed) << endl; // prints 8  
}
```

Python Օրինակ

```
def obf_add(packed):
    a = (packed >> 16) & 0xFFFF
    b = packed & 0xFFFF
    return a + b

packed = (5 << 16) | 3 # = 327683
print(obf_add(packed)) # prints 8
```

Մեթոդ 3 — Variadic Functions (Փոփոխական Արգումենտներ)

C/C++-ում ... (variadic) ֆունկցիան կարող է ընդունել ցանկացած թվով արգումենտ:
Reverse engineer-ը տեսնում է «count, ...» ստորագրությունը, բայց ոչ ընդհատ
պարզ է, թե ինչ արժեքներ են կանչի ժամանակ փոխանցվում:

C++ Օրինակ

```
#include <cstdarg>

int obf_add(int count, ...) {
    va_list args;
    va_start(args, count); // Start reading after 'count'

    int a = va_arg(args, int); // First hidden arg
    int b = va_arg(args, int); // Second hidden arg

    va_end(args);
    return a + b;
}

int main() {
    cout << obf_add(2, 5, 3) << endl; // prints 8
    // count=2 means '2 args follow', but analyst must guess
```


ԹԵՄԱ 2 — Կոդի Ցրում Հիշողության Տարբեր Հատվածներում

(Code Scattering in Memory)

Ի՞նչ է սա

Կոդը չի պահվում մեկ տեղ, այլ բաժանվում է մասերի և տեղադրվում հիշողության տարբեր հատվածներում: Կատարման ժամանակ ծրագիրը «հավաքում» է այդ մասերը ու կատարում: Արդյունքում դժվարանում է ստատիկ վերլուծությունը (disassembly):

Ե՞րբ է կիրառվում

- Packers — ծրագրի «փաթեթավորիչներ» (UPX, Themida)
- Malware loaders — վնասակար ծրագրի բեռնիչներ
- Obfuscated software — պաշտպանված ծրագրեր
- Shellcode injection — շպիտ կոդի ներարկում

Մեթոդ 1 — Function Pointer Scattering

Կոդի տարբեր հատվածներ (ֆունկցիաներ) կանչվում են ոչ անմիջականորեն, այլ pointer-ների (ցուցիչների) միջոցով: Pointer-ները կարող են պահվել array-երում, struct-երում, կամ heap-ում — ամենուր, ոչ թե կոդի հատվածում:

C++ Օրինակ

```

int part1(int x) { return x + 1; } // Stored in code segment
int part2(int x) { return x * 2; } // Stored in code segment

int main() {
    // Pointers stored in stack - scattered!

    int (*f1)(int) = part1;
    int (*f2)(int) = part2;

    // Logic: f2(f1(5)) = f2(6) = 12
    int result = f2(f1(5));

    cout << result << endl; // prints 12
}

```

Ինչ է կատարվում step-by-step:

1. f1 = pointer to part1 (address, e.g., 0x100003f20)
2. f2 = pointer to part2 (address, e.g., 0x100003f30)
3. f1(5) -> part1(5) -> 5 + 1 = 6
4. f2(6) -> part2(6) -> 6 * 2 = 12

Disassembler-ը տեսնում է CALL rax (indirect call), ոչ թե CALL part1

Python Օրինակ

```

def part1(x): return x + 1
def part2(x): return x * 2

# Functions stored in a list (like array of pointers)
functions = [part1, part2]

# Indirect call: functions[1](functions[0](5))
result = functions[1](functions[0](5))
print(result) # prints 12

```

Մեթոդ 2 — Dynamic Code Execution

Կողմը ստեղծվում է ծրագրի աշխատանքի ընթացքում (runtime) և կատարվում անմիջապես: C++-ում դա արվում է machine code-ի բեռնմամբ հիշողության մեջ (shellcode), Python-ում `exec()`-ի օգնությամբ:

C++ Օրինակ — Shellcode Execution

```
unsigned char code[] = {  
    0xB8, 0x05, 0x00, 0x00, 0x00, // MOV EAX, 5  
    0xC3 // RET  
};  
  
// On macOS/Linux: malloc + mprotect to make executable  
// On Windows: VirtualAlloc with PAGE_EXECUTE_READWRITE  
void* exec = VirtualAlloc(0, sizeof(code),  
    MEM_COMMIT, PAGE_EXECUTE_READWRITE);  
memcpy(exec, code, sizeof(code));  
  
int (*func)() = (int(*)())exec; // Cast to function pointer  
cout << func() << endl; // Executes shellcode! prints 5
```

x86 Assembly բացատրություն:

0xB8 = MOV EAX, imm32 (EAX = 5)

0x05 0x00 0x00 0x00 = 5 (little-endian 32-bit)

0xC3 = RET (վերադարձ)

Ֆունկցիան ուղղակի վերադարձնում է 5-ը EAX ռեգիստրի միջոցով

Disassembler-ը չի «տեսնում» այս կողմը compile-time-ին

Python Օրինակ — exec() Scattering

```
# Code is split into 3 separate strings (scattered)

code_part1 = 'x = 5'

code_part2 = 'y = x + 2'

code_part3 = 'print(y)'


# Each part is executed separately — order matters!

exec(code_part1)  # x = 5

exec(code_part2)  # y = 7

exec(code_part3)  # prints 7
```

Բոլոր exec() կանչերը կհսում են նույն namespace-ը, ուստի code_part1-ում ստեղծված x-ը հասանելի է code_part2-ում: Static analyzer-ը չի կարող հետևել կոդի հոսքին (control flow-ին), քանի որ code_part1..3-ը կարող են գաղտնագրված լինել ու բաժանված ծրագրում:

ԱՄԲՈՂՋԱԿԱՆ ԼՈՒԾՈՒՄՆԵՐ — ԿԱՏԱՐՄԱՆ ԱՐԴՅՈՒՆՔՆԵՐ

C++ — Բոլոր 3 Թեմա 1 Մեթոդները

```

#include <iostream>

#include <cstdarg>

using namespace std;


// Method 1: Dummy parameters
int obf_add_dummy(int a, int b, int fake1, int fake2) {
    return a + b; // fake1, fake2 ignored
}


// Method 2: Packed parameters
int obf_add_packed(int packed) {
    int a = (packed >> 16) & 0xFFFF;
    int b = packed & 0xFFFF;
    return a + b;
}


// Method 3: Variadic
int obf_add_variadic(int count, ...) {
    va_list args;
    va_start(args, count);
    int a = va_arg(args, int);
    int b = va_arg(args, int);
    va_end(args);
    return a + b;
}


// Topic 2: Function pointer scattering
int part1(int x) { return x + 1; }
int part2(int x) { return x * 2; }

int main() {
    // Topic 1

```

Կատարման Արդյունքներ (Output)

=== TOPIC 1: Argument Count Obfuscation ===

1. Dummy params obf_add(5, 3, 999, 12345) = 8
2. Packed params obf_add(327683) = 8
3. Variadic obf_add(2, 5, 3) = 8

=== TOPIC 2: Code Scattering in Memory ===

1. Function pointer dispatch f2(f1(5)) = 12
2. Dynamic exec result y = 7

Python — Ամբողջական Ծրագիր

```
# Topic 1 - Method 1: Dummy params
def obf_add_dummy(a, b, *args): # *args ignored
    return a + b
print(obf_add_dummy(5, 3, 999, 888, 777)) # 8
```

```
# Topic 1 - Method 2: Packed
def obf_add_packed(packed):
    a = (packed >> 16) & 0xFFFF
    b = packed & 0xFFFF
    return a + b
packed = (5 << 16) | 3
print(obf_add_packed(packed)) # 8
```

```
# Topic 2 - Method 1: Function dispatch
def part1(x): return x + 1
def part2(x): return x * 2
functions = [part1, part2]
print(functions[1](functions[0](5))) # 12
```

```
# Topic 2 - Method 2: exec() scattering
exec('x = 5')
exec('y = x + 2')
exec('print(y)') # 7
```

ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

Ինչ Սովորեցինք

Մեթոդ	Ինչ է անում	Նպատակ
Dummy Params	Կեղծ արգ. ավելացնում	Շփոթեցնել վերլուծողին
Packed Params	Բազմ. արժ. մեկ int-ում	Թաքցնել տվյալ. կառուցվ.
Variadic (...)	Ցանկ. թիվ արգ.	Թաքցնել ստ. ստ.
Func Pointers	Indirect CALL	Խանգ. disassembly
Dynamic Exec	Runtime code gen	Թաքցնել execution flow

Կազմված է MacBook Air M4-ով, C++ և Python ծրագրավորման լեզուներով: